

CampusEats

Team ID: 6

Team Members

Name	Roll Number
Gauraansh Gaur	20251651039
Shraiyansh Chaware	20251651085
Utkarsh Singh	20251651099
Abhishek Yadav	20251651007
Jatin Mandwani	20251651048

Context

The partner integrated here is UniPay, the campus bank's payment gateway. The one operation consumed is charge, which CampusEats Orders calls once a student confirms an order so the student's card is billed before the order is accepted.

This edge is built on SOAP while the rest of CampusEats stays REST because a bank will not accept a loosely typed request. UniPay publishes a rigid WSDL so every field, type and fault is fixed in advance, which rules out malformed charge attempts reaching a real card. Money movement also needs guarantees that plain REST does not give out of the box: WS Security signs and encrypts the message itself rather than only the transport, and the WS* stack is what the bank relies on for transactional, all or nothing behaviour on a charge. CampusEats therefore has no real choice on this edge, UniPay decided the contract and CampusEats simply has to speak it correctly.

HTTP Binding

The charge envelope shown in soap request.xml is carried as the body of a plain HTTPS POST. The method, host and SOAPAction below come straight from the binding in partner.wsdl and the path comes from the service and port entry.

POST /pay HTTP/1.1

Host: api.unipay.example

Content-Type: text/xml; charset=utf-8

SOAPAction: "urn:unipay:charge"

Content-Length: 412

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:uni="urn:unipay:payments">
  <soap:Header>
    <uni:auth token="CE-MERCHANT-KEY-4471"/>
  </soap:Header>
  <soap:Body>
    <uni:charge>
      <cardToken>tok_9f2c81a</cardToken>
      <amount>420.00</amount>
      <currency>INR</currency>
      <orderRef>CE-ORD-10245</orderRef>
    </uni:charge>
  </soap:Body>
</soap:Envelope>
```

UniPay answers on the same connection with a 200 OK carrying the chargeResponse envelope from soap response.xml when the card is approved, or with a 500 Internal Server Error carrying the soap:Fault from soap fault.xml when it is declined.

Discovery

UniPay's WSDL is obtained the way most bank integrations actually happen today: the bank's integration team sends CampusEats a direct link to the WSDL, together with a merchant key, once the partnership paperwork is signed.

There is no live registry server for CampusEats to query at call time. What plays the role of UDDI instead is a small internal catalogue that CampusEats keeps for every partner it integrates, listing where each partner's contract lives so the next developer does not have to go hunting for it.

The single record for UniPay in that catalogue is shown below.

Field	Value
Business	UniPay Bank Ltd.
Service	Payments
Endpoint	https://api.unipay.example/pay
Specification	urn:unipay:payments, described by partner.wsdl

UniPay publishes the WSDL and hands over the catalogue entry, CampusEats finds it by looking the entry up once during integration setup, and every charge call afterwards binds directly to the endpoint with no registry involved.

Fault Mapping

UniPay's fault is written in the bank's own vocabulary and must never reach a CampusEats student directly. When Orders receives a soap:Fault whose detail carries code equal to card_declined, it catches that fault at the boundary and raises the payment declined error that the placeOrder operation already promised in the Assignment 2 contract. The student only ever sees payment declined; they never see the words soap:Fault, card_declined or UniPay anywhere in the response.

The mapping used by Orders is kept small and explicit so it is easy to extend later

UniPay fault code	CampusEats error (placeOrder)
card_declined	payment declined
gateway_unavailable	payment temporarily unavailable
invalid_card_token	payment method invalid